

Day 6: Project Reframe, Two-Track Plan and Repository Consolidation

Date: 6 July 2026

1. Aim of Today's Work

Today was a planning and consolidation day. Following a team meeting that clarified the project's direction, the whole project was reframed into two clear tracks. Alongside that, the team's GitHub repository and all its documentation were consolidated and made consistent, so that the base is solid before hands-on work begins. No new hardware was built today, the focus was on getting the plan and the project's foundations right.

2. Project Direction

The clarified end goal of the project is to take a regular, non-autonomous car and, through research and development, work toward giving it self-driving capability, climbing the levels of vehicle autonomy one by one. The aim is to implement each level properly, working toward full self-driving as the end goal.

Two concrete priorities were set, and the project is now organised around them:

- A standardized way of acquiring vehicle data: defining how data is obtained, formatted, stored, and streamed, before testing it on a real vehicle.
- A self-driving proof-of-concept: demonstrating, on a model car we build, how that data can be used to implement self-driving techniques and collision avoidance.

3. The Two-Track Plan

3.1 Track 1: Data Acquisition Standard

A proper, standardized, and efficient method for acquiring and streaming vehicle data. The design comes first, then the implementation, then validation on a real car.

- OBD-II data read in-house through an ELM327 adapter wired directly to the ESP32 (not via a third-party phone app), packed into the defined 32-byte vehicle packet.
- Tyre data from our own aftermarket 433 MHz TPMS sensors and receiver, packed into the defined 16-byte tyre packet.
- Data streamed wirelessly to the backend and made available to multiple devices at once.
- A vehicle-research deliverable: identifying which vehicles have clean, standard-compliant OBD-II support suitable for our data acquisition.

3.2 Track 2: Self-Driving Model Car

A model car we build ourselves, demonstrating autonomous navigation in a bounded environment, climbing the autonomy levels one by one. Parts are chosen for capability and the demonstration goal.

- A* pathfinding with the ability to re-route and find the shortest path when the situation changes.
- Collision detection and avoidance: sensing an obstacle, and re-planning the path around it.
- A capable compute setup: a Raspberry Pi as the autonomy “brain” with an ESP32 as the motor/CAN controller underneath it (an alternative with the processing done on a separate device is also being considered).
- An honest level ladder for a bounded model: from single-function assistance up to full self-driving within the arena with obstacle re-routing.

3.3 Parked for later

Two ideas raised in the meeting are deliberately set aside until the two tracks above are underway: fleet/multi-vehicle tracking (a live map of all vehicles, with drill-down into any one), and a digital-twin visualisation.

4. Repository & Documentation Consolidation

With the plan set, the team's GitHub repository and its documentation were cleaned up and made consistent, so nothing contradicts and the base is reliable for the work ahead. This included:

- A repository history cleanup, removing files that should never have been version-controlled (committed database files and other artifacts) from the project's history.
- Adding a .gitignore so those kinds of files are never committed again.
- Reconciling the deployment URLs, which had disagreed across several documents, to one consistent, correct set.
- Consolidating duplicated documents into a single canonical location, so there is only ever one copy of each specification.
- Documenting a known drift between the shared data schema and the code (fields already in use but not yet formally defined), so the gap is visible and tracked, pending a decision as part of the upcoming data-standard work.
- Correcting documents that no longer matched reality: the backend brief now reflects that the backend is built and deployed, and the earlier proof-of-concept parts list is clearly marked as a superseded, historical decision.

A final review of the repository was carried out and confirmed everything is consistent and in order.

4.1 Repository Governance Now in Place

Beyond the one-off cleanup, the team now works out of a properly governed GitHub organisation and repository: one folder per track, branch protection on the main branch (all changes go through reviewed pull requests, with no direct pushes), and an automated daily activity log that records each day's commits into a dated report for the team to add notes to. This gives the project proper version control, review discipline, and a consistent day-by-day record going forward.

5. Dashboard Update: Control Console Redesign

The Vehicle Control Console (Dashboard 2) was redesigned with a cleaner, clearer layout. It now shows the drive state, speed, distance-to-target, obstacle distance, and battery as status cards along the top, a live telemetry chart, the full set of drive controls (start/stop, forward/back/left/right, brake, emergency stop, and a target-speed slider), and a recent-activity log for outbound commands and inbound status. It connects to the backend at `wss://api.nalusa.space/ws`.

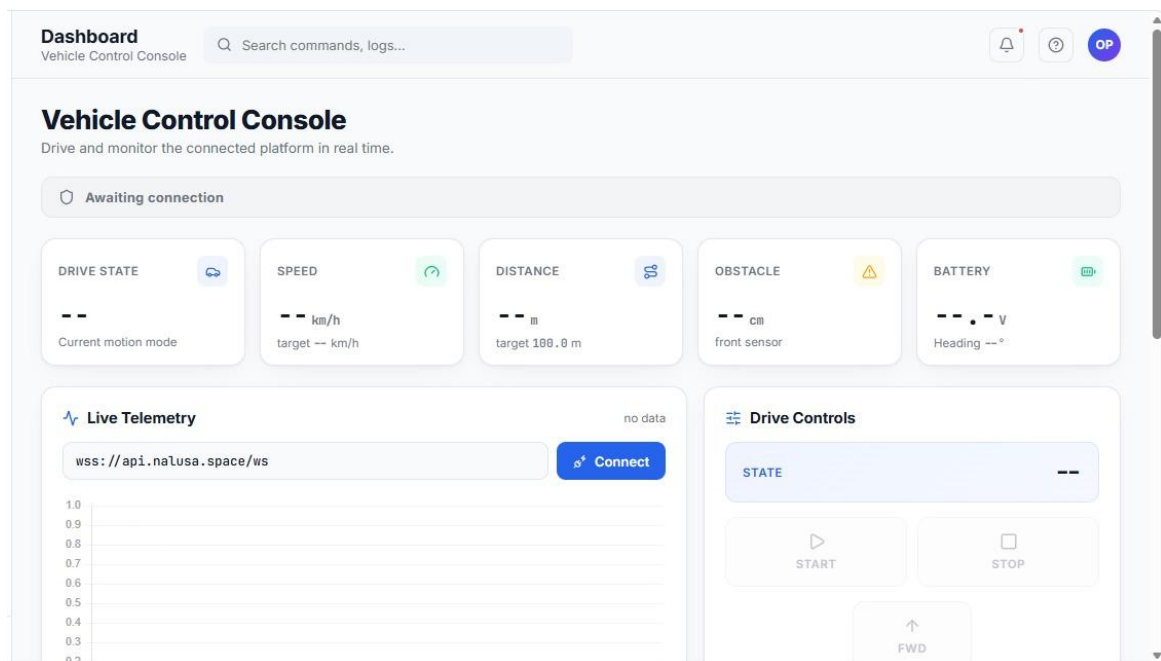


Figure: Redesigned Vehicle Control Console with status cards, live telemetry, and drive controls

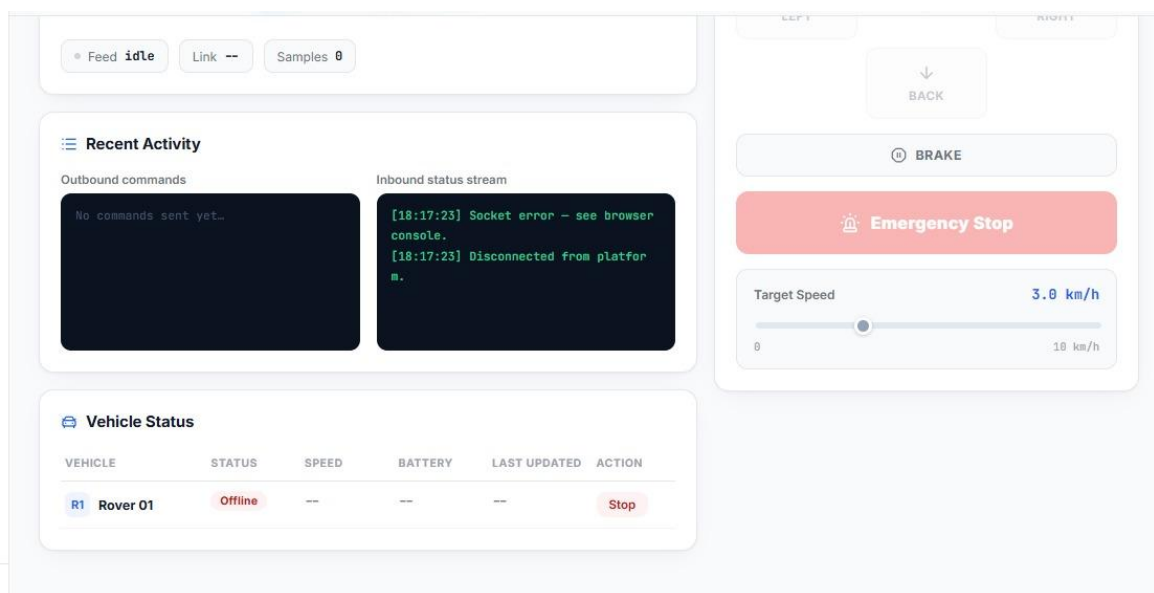


Figure: Lower half showing recent-activity command/status logs, drive controls, and a Vehicle Status table (an early nod to future multi-vehicle support)

6. Next Phase: Moving to Hands-On Work

With the plan and the repository in order, the project now moves from planning into hands-on work. Two things happen in parallel next:

6.1 Finalising the data output design (Track 1)

Deciding and documenting exactly what data we will get and how it moves, so that real data can be collected as soon as the hardware is in hand:

- What data: which OBD-II parameters (PIDs) make up the 32-byte vehicle packet, and which tyre values make up the 16-byte TPMS packet.
- How it is received: in-house, reading the OBD-II line through an ELM327 adapter wired directly to the ESP32.
- How it is formatted and stored: the standardized packet format, stored on the backend.
- How it is sent: wirelessly to the backend and out to multiple devices.

6.2 Parts procurement (both tracks)

Both tracks' parts lists are to be confirmed tomorrow so that hands-on work and real data collection can begin immediately after:

- Track 1: the OBD-II acquisition hardware and cabling, and our own 433 MHz TPMS sensors and RF receiver.
- Track 2: the self-driving platform (Raspberry Pi + ESP32, encoder motors and driver, a sensor for localization and obstacle detection, IMU, chassis, and battery). This list is finalised tomorrow.

Sourcing: parts can be bought locally from Mercy Electronics (Tambaram) and Swastik Electronics (near VIT), and online from Robu.in, Robocraze, ThinkRobotics, Quartz Components, and Amazon.in for items not available locally.

7. Next Steps

- Finalise the data output design (what data, how it is received, formatted, stored, and sent) and the work split.
- Confirm both tracks' parts lists tomorrow and purchase what is needed to begin hands-on work.
- Begin real data acquisition for Track 1 and the platform build for Track 2 once parts are in hand.